ANSH Friend ZIP Integration Plan

Date: May 2, 2026
Context: Your friend built a website (ZIP). You have Python scripts on your laptop. This document tells you exactly what to keep, what to change, and how to connect both pieces.

PART 1: What Your Friend Actually Built

The Verdict: A LOT. Way more than scaffolding.

Your friend built a **full frontend architecture** with:

What	Status	Your Action
React 19 + Vite	✅ Clean setup	Keep as-is
Gold-on-black theme	✅ Premium, stealthy	**Keep. Better than saffron for Phase 1.**
22 ślokas database	✅ Full Sanskrit, meaning, context, tags	**Keep. Expand with recordings later.**
Typography (Cormorant + Inter + Noto Devanagari)	✅ Perfect	Keep as-is
Hero section	✅ Massive ANSH title, gradient, scroll hint	Keep, tweak tagline
Navbar	✅ Fixed, blur-on-scroll, mobile hamburger	Keep as-is
Listen page	✅ Track listing + waveform bars + play button	**Modify: connect to WAV files**
Read page	✅ Shloka cards with expand/collapse + bookmarks	Keep, add acoustic features later
Study page	✅ Learning section structure	Keep, fill with meter tutorials
Mission page	✅ 3 pillars + 2 doors layout	Keep, tweak copy to match thesis
Process page	✅ Step-by-step timeline	**Modify: map to thesis timeline**
Community	✅ Layout exists	Lock behind 19-shloka gate
SearchBar	✅ Functional	Keep as-is
ShlokCard	✅ Card + row variants	Keep both, add reciter badge
Footer	✅ Present	Keep as-is
useBookmarks hook	✅ localStorage persistence	**Keep. Perfect for favorites.**
useSpeech hook	✅ Text-to-speech Sanskrit + English	**Keep. Great accessibility.**
useAudioDrone hook	✅ Web Audio API synthesizer	**Replace with WAV player**

What They DIDN'T Build (What You Need to Add)

Missing Piece	Why It Matters	Effort
9-Shloka Gate mechanic	Core engagement system, thesis data collection	Medium
WAV audio player	Currently plays synthesized drones, not recordings	Medium
Acoustic feature display	Shows Python pipeline output on recording pages	Low
Reciter type badges	Devotional vs non-devotional indicator	Low
Listener rating interface	1-7 Likert scale for emotion ratings	Medium
Admin upload panel	You upload WAVs + trigger Python analysis	Medium
Backend API	For Phase 2. Not needed now.	Future

PART 2: Theme Decision — Gold vs Saffron

Your Friend's Theme: Gold (#c9a84c)

```
Background:  #080808 (near-black)
Text:        #f5f5f0 (warm white)
Accent:      #c9a84c (gold)
Light accent:#e8c96e (light gold)
Dim accent:  rgba(201,168,76,0.12)
```

My Recommended Theme: Saffron (#FF9933)

```
Background:  #0A0A0A (near-black)
Text:        #F5F0E8 (warm white)
Accent:      #FF9933 (saffron)
Light accent:#FFB366 (light saffron)
```

The Decision: KEEP GOLD. It's better for Phase 1.

Gold	Saffron
Reads as "premium archive"	Reads as "Hindu spiritual"
Stealth —doesn' t trigger religious associations	Explicitly signals Indian/Hindu identity
Warm, temple-like, sophisticated	Bright, energetic, political
Your friend already built it perfectly	Would require full CSS rewrite

Gold is the right choice for a stealth academic archive. Saffron can be introduced later as a secondary accent for specific elements (thesis data highlights, emotion rating active states).

PART 3: Python Pipeline Integration

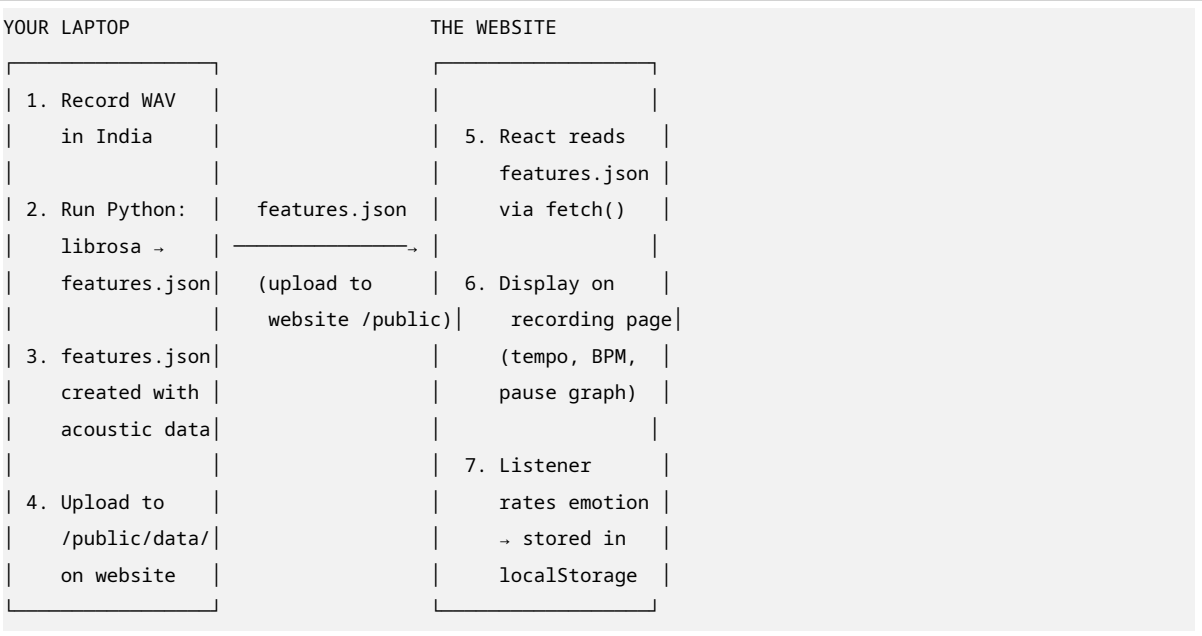
Your Laptop Has:

- Python scripts using **librosa**, **Praat**, **Essentia**
- Scripts that extract: tempo, pause architecture, accent density, meter type, F0 contour, formants, MFCCs
- These run locally on your machine

Your Friend's Frontend Has:

- React components that display track listings
- Waveform visualization (currently fake/generated)
- No connection to any audio analysis data

How They Connect:



The Data Format (What Python Outputs, React Reads)

Your Python script should output a JSON file like this:

```
{
  "recordingId": "BG_2.20_VNS_PanditR_Slow",
  "features": {
    "tempoBPM": 68,
    "pauseArchitecture": [0.3, 0.5, 0.3, 1.2],
    "accentDensity": 12,
    "meterType": "anuṣṭubh",
    "rhythmicRegularity": 0.92,
    "f0Contour": [120, 122, 118, 125, 121],
    "formants": [750, 1200, 2500],
    "mfccs": [[0.1, 0.2, ...], [0.15, 0.22, ...]]
  },
  "emotionProfile": {
```

```
    "calm": 6.2,
    "awe": 4.1,
    "absorption": 5.8,
    "tension": 2.1,
    "sacredness": 5.5
  }
}
```

For Phase 1: These JSON files live in `/public/data/` as static files. The React frontend fetches them with a simple `fetch('/data/BG_2.20.json')`.

No backend needed yet. Static JSON is fine for 60-120 recordings.

PART 4: Component-by-Component Integration Plan

KEEP AS-IS (No Changes Needed)

Component	Why Keep	Notes
`index.css` (theme)	Perfect gold-on-black system	Just works
`Hero.css` + Hero component	Stunning landing	Update tagline text only
`Navbar.css` + Navbar	Clean, functional, mobile-ready	Keep all links
`Mission.css` + Mission	3 pillars = Listen, Read, Study	Tweak pillar copy
`Process.css` + Process	Timeline layout	Map steps to thesis phases
`Footer.css` + Footer	Present	Keep
`SearchBar.css` + SearchBar	Functional	Keep
`ShlokCard.css` + ShlokCard	Card + row variants	Add reciter badge
`useBookmarks.js`	localStorage favorites	Keep as-is
`useSpeech.js`	TTS for Sanskrit + English	Keep as-is
`shlokas.js`	22 ślokas with full data	**Keep. This is gold.**

MODIFY (Needs Changes)

Component	What to Change	How
`useAudioDrone.js`	Currently synthesizes tones	**Replace with WAV audio player using HTML5 Audio API or Howler.js**
`Listen.css` + Listen page	Shows track list with fake waveform	**Connect to real WAV files. Display real waveform from Python output.**
`Read.css` + Read page	Text-only shloka display	**Add acoustic feature panel (from features.json)**
`Study.css` + Study page	Generic learning layout	**Fill with Sanskrit meter tutorials**
`Community.css` + Community	Open access	**Lock behind 19-shloka gate**
`Founder.css` + Founder	Your bio section	**Update with Ashoka + thesis info**

BUILD NEW (Missing Pieces)

New Component	Purpose	Effort
`NineShlokaGate.jsx + .css`	The core engagement mechanic	1 day
`AcousticPanel.jsx + .css`	Displays Python features on recording page	4 hours
`ReciterBadge.jsx`	Small badge: devotional (green) vs non-devotional (rose)	2 hours
`EmotionRating.jsx + .css`	1-7 Likert scale for listener ratings	4 hours
`AdminUpload.jsx + .css`	Upload WAV + trigger Python + view status	1 day
`WaveformPlayer.jsx`	Real waveform from audio file + play/pause	4 hours

PART 5: Priority Build Order

Week 1: Foundation

1. ✅ **Keep all friend's CSS and components**
2. ✅ **Keep `shlokas.js` data** (22 ślokas)
3. 🔧 **Replace `useAudioDrone.js`** with real WAV player
4. 🔧 **Build `WaveformPlayer.jsx`** — reads audio file, shows real waveform
5. 🔧 **Connect Listen page** to play actual audio files

Week 2: The Gate

6. 🔧 **Build `NineShlokaGate.jsx`** — the 9-recording unlock mechanic
7. 🔧 **Add gate to archive access** — must complete 9 before full browse
8. 🔧 **Track completion** in localStorage + state

Week 3: Thesis Integration

9. 🔧 **Build `AcousticPanel.jsx`** — reads features.json, displays tempo, BPM, pauses
10. 🔧 **Add `ReciterBadge.jsx`** — green for devotional, rose for non-devotional
11. 🔧 **Update Process page** — map to your thesis timeline (Monsoon 2026 → Spring 2028)
12. 🔧 **Build `EmotionRating.jsx`** — 1-7 Likert, stores in localStorage for now

Week 4: Polish

13. 🔧 **Update Mission page** — thesis-specific copy (acoustic analysis, meter study)
14. 🔧 **Update Founder page** — your bio: "Ayush Bhoi, Ashoka University"
15. 🔧 **Build `AdminUpload.jsx`** — you upload WAVs, system stores them
16. 🎨 **Fine-tune** — mobile responsiveness, animation timing, edge cases

---

PART 6: Your Friend's Code Structure

Based on the CSS files, here's the component map:

```
src/
├── main.jsx              ← Entry point (keep)
├── App.jsx               ← Router, layout (modify: add gate logic)
├── index.css             ← Theme variables (keep as-is)
├── App.css               ← Counter + page transitions (keep counter, remove Vite boilerplate)
│
├── data/
│   └── shlokas.js        ← 22 ślokas (keep, expand later)
│
├── hooks/
│   ├── useAudioDrone.js  ← 🔴 REPLACE with WAV player
│   ├── useBookmarks.js   ← ✅ Keep as-is
│   └── useSpeech.js      ← ✅ Keep as-is
│
├── components/
│   ├── Navbar.jsx        ← ✅ Keep, update links
│   ├── Hero.jsx          ← ✅ Keep, update tagline
│   ├── Listen.jsx        ← 🔴 MODIFY: real audio
│   ├── Read.jsx          ✅/🔴 Keep, add acoustic panel
│   ├── Study.jsx         ✅/🔴 Keep, fill content
│   ├── Mission.jsx       ✅/🔴 Keep, tweak copy
│   ├── Process.jsx       ✅/🔴 Keep, map to thesis
│   ├── Community.jsx     🔴 MODIFY: add 19-shloka gate lock
│   ├── Founder.jsx       ✅/🔴 Keep, update bio
│   ├── Footer.jsx        ← ✅ Keep
│   ├── SearchBar.jsx     ← ✅ Keep
│   ├── ShlokCard.jsx     ← ✅ Keep, add reciter badge
│   └── Rights.jsx        ← ✅ Keep
│
├── pages/
│   └── Home.jsx          ← Assembles all sections (keep structure)
│
└── assets/
    └── hero.png          ← Hero background image (keep)
```

PART 7: How to Test the Integration

Step 1: Run Friend's Code

```
cd ansharchive-website
npm install
npm run dev
```

Step 2: Create a Test Recording

1. Record yourself saying one Gita verse (phone is fine)
2. Save as WAV file
3. Put in `/public/recordings/test.wav`

Step 3: Run Your Python Pipeline

```
python extract_features.py --input test.wav --output test.json
```

4. Copy `test.json` to website's `/public/data/test.json`

Step 4: Connect Frontend to Data

- Modify Listen page to load `test.wav`
- Modify Read page to display `test.json` features
- Verify: when you click play, real audio plays
- Verify: when you view the shloka, acoustic features display

Step 5: Test 9-Shloka Gate

- Create 9 short test audio files
- Set up gate sequence
- Verify: must listen to all 9 before archive unlocks

PART 8: The One-Pager for Your Friend

Text this to your friend:

> "Bro, I analyzed your ZIP. It's way better than I expected. Keep the gold theme — it's perfect. Keep all the CSS, all the components, all the shloka data. Here's what I need you to change:
>
> 1. Replace the fake audio with real WAV playback (I have recordings)
> 2. Build a 9-shloka gate: visitor must listen to 9 verses before archive unlocks
> 3. Add an 'acoustic features' panel on each recording page that reads JSON data from my Python scripts (tempo, BPM, pause graph)
> 4. Add a small badge on each recording: green = devotional reciter, red = non-devotional
> 5. Add a 1-7 rating slider for emotion (calm, awe, absorption) on each recording
>
> My Python pipeline outputs JSON files with acoustic data. The React frontend just reads those JSON files from /public/data/. No backend needed.
>
> Priority order: (1) real audio player, (2) 9-shloka gate, (3) acoustic panel, (4) reciter badge, (5) emotion rating.
>
> The gold theme stays. Don't change any colors. Just add the new features."

PART 9: Theme Comparison Visual

Element	Friend' s Gold	My Saffron	Winner
Background	#080808	#0A0A0A	Friend' s (darker, more premium)
Accent	#c9a84c (muted gold)	#FF9933 (bright saffron)	Friend' s (less political)
Text	#f5f5f0 (warm white)	#F5F0E8 (warm white)	Tie
Hover state	#e8c96e (light gold)	#FFB366 (light saffron)	Friend' s (more subtle)
Card bg	#1a1a1a	#141414	Friend' s (already built)
Scrollbar	Gold thumb	Saffron thumb	N/A

Table 7 – continued

Element	Friend' s Gold	My Saffron	Winner
Overall feel	Premium archive, scholarly	Energetic, political	**Friend' s for Phase 1**

Decision: Keep gold. Introduce saffron ONLY as a secondary accent for thesis-specific elements (active rating state, data highlights).

PART 10: Next Steps

1. **Show this document to your friend**
2. **Friend reads the ZIP structure** (they know it already)
3. **Friend implements Priority 1** (real audio player) — 1 day
4. **You test with one real recording**
5. **Friend implements Priority 2** (9-shloka gate) — 1 day
6. **You test the gate flow**
7. **Continue down the priority list**

Don't rewrite anything. Build on top of what exists. The foundation is solid.

Document Version: 1.0 — May 2, 2026